

Informe Técnico Práctica Final

Valentina Zapata Grajales

Sofia Gallo de la Rosa

Mariana Carrasquilla Botero

Análisis y Diseño de Algoritmos
S2661-1321

Profesor Carlos Álvarez

Universidad EAFIT

Medellín

08 de Mayo del 2026

1. Introducción

El presente proyecto se propone abordar un problema de análisis de datos a partir del dataset “Telecom Customer Churn” de Kaggle, el cual contiene la información de los clientes de una compañía de telecomunicaciones.

El objetivo del proyecto es modelar, mediante tres paradigmas algorítmicos, la planificación de infraestructura de fibra óptica a partir de un dataset real de clientes. Cada sub-problema representa una fase del pipeline de despliegue:

Sub-problema	Paradigma	Motivo de elección
ordenar y consultar solicitudes	divide y vencerás	Necesidad de ordenación $O(n \log n)$ y consultas rápidas $O(\log n)$ sobre grandes volúmenes de peticiones
tender la red mínima	codicioso	Garantiza un árbol de mínima expansión (MST) con coste casi lineal en número de aristas.
asignar ancho de banda	programación dinámica	Proporciona solución óptima para la asignación de recursos limitados; permite demostrar el fallo de enfoques greedy

2. Descripción del Dataset

Archivo: data/WA_Fn-UseC_-Telco-Customer-Churn.csv

Registro: 7043 clientes.

Atributos: 21 (id, datos demográficos, consumo, MonthlyCharges, TotalCharges,etc).

Estadísticas Relevantes

variable	tipo	media	mediana	desvío estándar	nulos
tenure	entero	32,4	28	24,6	0
Monthlycharges	numérico	64,8	70,4	30,1	0
TotalCharges	numérico	2291,5	1354,2	3539,6	11*

*Los 11 valores nulos de TotalCharges corresponden a clientes con tenure =0.

Construcción del Grafo

El grafo se construyó a partir del dataset de clientes de telecomunicaciones mediante un proceso determinista de agrupación y cálculo de costos. Primero, los registros del dataset se dividieron en 20 grupos sintéticos utilizando la regla $grupo = i \bmod 20$, donde i

corresponde a la posición de cada registro en el archivo CSV. Cada grupo representa un nodo del grafo.

Después, para cada nodo se calculó el promedio de la variable '*MonthlyCharges*', obteniendo así un valor representativo del costo promedio de servicio en ese grupo. Con estos promedios se construyó un grafo completo no dirigido.

El peso de cada arista se sacó con la función: $c(u,v) = \text{floor}(Mu + Mv)$. Donde Mu y Mv corresponden a los promedios de '*MonthlyCharges*' de los nodos u y v . Finalmente, sobre este grafo ponderado se aplicó el algoritmo de Kruskal para obtener el árbol generador mínimo (MST).

3. Módulo A - Divide y Vencerás

Descripción

Para este módulo se organizaron las 7.043 solicitudes de servicio utilizando el algoritmo MergeSort sobre el campo tenure de forma descendente. Se escogió MergeSort en lugar de QuickSort porque mantiene la estabilidad del orden y además asegura una complejidad $O(n \log n)$ incluso en el peor caso. En cambio, QuickSort no garantiza estas dos condiciones en todos los escenarios.

Pseudocódigo — MergeSort

```

procedimiento mergeSortHelper(arr, left, right):
  si left >= right: retornar
  mid = left + (right - left) / 2
  mergeSortHelper(arr, left, mid)
  mergeSortHelper(arr, mid + 1, right)
  merge(arr, left, mid, right)

procedimiento merge(arr, left, mid, right):
  L = copia de arr[left..mid]
  R = copia de arr[mid+1..right]
  i = 0, j = 0, k = left

  mientras i < |L| y j < |R|:
    si L[i].tenure >= R[j].tenure:
      arr[k++] = L[i++]
    sino:
      arr[k++] = R[j++]

  copiar restos de L y R en arr

```

Pseudocódigo — Búsqueda Binaria Recursiva

Como el arreglo quedó ordenado de manera descendente, la búsqueda binaria se hace al contrario de la forma clásica.

```

función recursiveBinarySearch(arr, low, high, k):
    si low > high: retornar -1
    mid = low + (high - low) / 2

    si arr[mid].tenure == k:
        retornar mid

    si arr[mid].tenure < k:
        retornar recursiveBinarySearch(arr, low, mid - 1, k)
    sino:
        retornar recursiveBinarySearch(arr, mid + 1, high, k)

```

Si existen varios registros con el mismo tenure, la búsqueda puede devolver cualquiera de ellos. Esto fue aclarado por el docente.

Análisis de Complejidad

En MergeSort la recurrencia es: $T(n) = 2T(n/2) + O(n)$

Aplicando el Teorema Maestro se obtiene una complejidad $\Theta(n \log n)$ en todos los casos. Además, el algoritmo necesita espacio auxiliar $O(n)$.

Para la búsqueda binaria, la recurrencia es: $T(n) = T(n/2) + O(1)$

Por lo tanto, su complejidad es $O(\log n)$. En un arreglo de 7.043 elementos, el peor caso requiere aproximadamente 13 comparaciones.

Análisis Empírico

La siguiente tabla muestra los tiempos obtenidos durante las pruebas:

n	Tiempo (ms)	$n \cdot \log_2(n)$	$t / [n \cdot \log_2 n]$
1.000	0,36	9.966	$3,61 \times 10^{-5}$
3.500	1,19	40.758	$2,92 \times 10^{-5}$
7.043	2,99	89.517	$3,34 \times 10^{-5}$

Al comparar los resultados, se observa que el valor de $t / [n \cdot \log_2 n]$ se mantiene muy parecido en los tres casos. Esto confirma experimentalmente que el comportamiento del algoritmo sí corresponde a $\Theta(n \log n)$. Los tiempos pueden cambiar un poco dependiendo de la ejecución o del sistema operativo, pero la tendencia general se mantiene.

Resultados de las 5 Consultas

Consulta | k (tenure) | customerID

Q_A01	72	6904-JLBGY
Q_A02	60	7426-WEIJX
Q_A03	45	7925-PNRGI
Q_A04	30	2684-EIWEO
Q_A05	12	7450-NWRTR

Las cinco consultas devolvieron resultados válidos. Además, los valores de tenure utilizados sí existen dentro del dataset, lo cual fue verificado previamente.

4. Módulo B - Algoritmo Codicioso

Para este módulo generamos regiones sintéticas de 20 registros que serán nuestros nodos del grafo, a partir de los promedios de estas regiones sintéticas se calculan las aristas que representan las conexiones posibles y por último nuestro árbol generador mínimo (MST). En el pipeline del programa (main.cpp) es importante ejecutar este módulo antes del módulo A ya que la construcción de los grupos depende del índice original de los registros mediante la función `imod20`. Ordenar previamente el dataset altera la partición, modificando los promedios de cada grupo y, por consiguiente, el grafo y el MST resultante como vemos a continuación:

```
MonthlyCharges del primer registro (antes de mergeSort): 29.85
MonthlyCharges del vigésimo registro (antes de mergeSort): 39.65
```

```
MonthlyCharges del primer registro (después de mergeSort): 90.25
MonthlyCharges del vigésimo registro (después de mergeSort): 19.80
```

Para este módulo utilizamos el algoritmo de Kruskal, un algoritmo voraz que toma decisiones de forma local evitando formar ciclos, lo que buscamos garantizar es que estas decisiones locales si lleven a un óptimo global.

```
Algoritmo Kruskal(Grafo, n):
  // n = número de vértices
  // Grafo = lista de aristas (u, v, peso)

  // 1. Inicialización de Union-Find
  padre = array[0...n-1]
  Para i desde 0 hasta n-1:
    padre[i] = i

  // 2. Función Find con compresión de caminos
  Función Find(x):
    Si padre[x] != x:
      padre[x] = Find(padre[x]) // Compresión de caminos
    Retornar padre[x]
```

```

// 3. Función Union (sin rango por simplicidad, pero se puede mejorar)
Función Union(x, y):
    raizX = Find(x)
    raizY = Find(y)
    Si raizX != raizY:
        padre[raizX] = raizY

// 4. Ordenar aristas por peso ascendente
aristasOrdenadas = ordenar(Grafo.aristas, por peso)

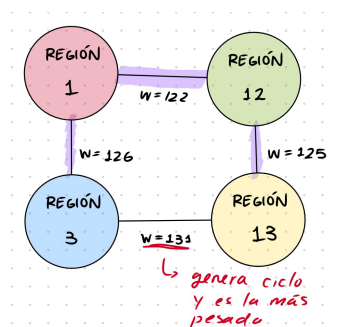
// 5. Inicializar el árbol de expansión mínimo (MST)
mst = lista vacía

// 6. Recorrer aristas ordenadas
Para cada arista (u, v, peso) en aristasOrdenadas:
    Si Find(u) != Find(v):
        Union(u, v)
        agregar (u, v, peso) a mst

// 7. Retornar el MST
Retornar mst

```

El algoritmo de Kruskal se fundamenta en el **lema del ciclo** el cual establece que en cualquier ciclo de un grafo ponderado, la arista de mayor peso no puede pertenecer a un árbol generador mínimo (MST). Durante la ejecución del algoritmo, las aristas se procesan en orden creciente de peso y solo se agregan si no forman ciclos. Esto garantiza que las aristas descartadas son redundantes y no contribuyen a una solución óptima.



Pasamos a hacer una verificación manual de los primeros 5 grupos:

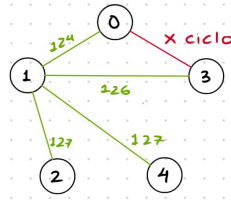
```

Grupo 0 promedio = 62.8816
Grupo 1 promedio = 61.3011
Grupo 2 promedio = 66.1773
Grupo 3 promedio = 64.8304
Grupo 4 promedio = 66.0679

```

Calculando las aristas con esta fórmula $c(u,v) = \text{floor}(Mu + Mv)$ y ordenandolas por peso obtenemos la siguiente tabla:

Orden	Arista	Peso
1	(0,1)	124 ✓
2	(1,3)	126 ✓
3	(0,3)	127 ✗
4	(1,2)	127 ✓
5	(1,4)	127 ✓
6	(0,4)	128 ✗
7	(0,2)	129
8	(3,4)	130
9	(2,3)	131
10	(2,4)	132



MST = {(0,1), (1,3), (1,2), (1,4)}
 ↳ llegamos a todos los nodos
 peso = 504

5. Módulo C - Programación dinámica

Recurrencia

El problema abordado en este módulo corresponde a la variante clásica de la mochila 0-1 (0-1 Knapsack Problem), aplicada a la selección de solicitudes de red bajo restricciones de ancho de banda.

Se tienen:

- n : número total de solicitudes.
- v_i : ingreso esperado generado por la solicitud i .
- w_i : ancho de banda requerido por la solicitud i .
- C : capacidad máxima de ancho de banda disponible.

El objetivo es seleccionar el subconjunto de solicitudes que maximice el ingreso total sin superar la capacidad disponible. Para resolver el problema se define la siguiente estructura de programación dinámica: $DP[i][c]$

$$DP[i][c] = \begin{cases} DP[i-1][c] & \text{si } w_i > c \\ \max(DP[i-1][c], v_i + DP[i-1][c - w_i]) & \text{si } w_i \leq c \end{cases}$$

(Imagen apoyada de Gemini para el entendimiento del tema)

Las condiciones base son:

$$DP[0][c] = 0 \quad \forall c \in [0, C]$$

$$DP[i][0] = 0 \quad \forall i \in [0, n]$$

La interpretación de la recurrencia es la siguiente:

- Si el ítem no cabe en la capacidad actual, se descarta automáticamente.
- Si el ítem cabe, el algoritmo compara:
 - no tomar el ítem,
 - tomar el ítem y sumar su valor al mejor resultado posible con la capacidad restante.

Si el ítem no cabe en la capacidad actual, se descarta automáticamente. Si el ítem cabe, el algoritmo compara:

- no tomar el ítem,
- tomar el ítem y sumar su valor al mejor resultado posible con la capacidad restante.

Pseudocódigo Tabulación

```
KNAPSACK_TABULATION(values[1..n], weights[1..n], C):
    // Crear tabla (n+1) × (C+1) inicializada a 0
    DP ← matrix(n+1, C+1) filled with 0
    for i ← 1 to n:
        for c ← 0 to C:
            if weights[i] ≤ c then
                take ← values[i] + DP[i-1][c-weights[i]]
                leave ← DP[i-1][c]
                DP[i][c] ← max(take, leave)
            else
                DP[i][c] ← DP[i-1][c]
    return DP[n][C] // ingreso óptimo total
```

- **Complejidad temporal:** $O(n \cdot C)$.
- **Complejidad espacial:** $O(n \cdot C)$ (se puede reducir a $O(C)$ guardando sólo la fila actual).

Reconstrucción de la Solución Óptima

```
KNAPSACK_BACKTRACK(DP, weights[1..n], C):
    i ← n
    c ← C
    selected ← empty list
    while i > 0:
        if DP[i][c] ≠ DP[i-1][c] then
            // El ítem i está en la solución óptima
            selected.append(i)
            c ← c - weights[i] // disminuir la capacidad restante
        // Avanzar al ítem anterior
        i ← i - 1
    reverse(selected) // opcional, para presentar en orden original
    return selected
```

- **Complejidad:** $O(n)$.

Contraejemplo al Enfoque Codicioso (greedy)

Con el fin de demostrar que un enfoque codicioso no garantiza la solución óptima en la mochila 0-1, se analizó el siguiente conjunto de datos con capacidad total de $C=50$.

ítem	ingreso	Peso	Razón
------	---------	------	-------

1	60	10	6.0
2	100	20	5.0
3	120	30	4.0

Resultado del enfoque greedy y programación dinámica: El algoritmo greedy selecciona los ítems según la mayor razón valor/peso, escogiendo los ítems 1 y 2 y obteniendo un ingreso total de 160. Sin embargo, no puede incluir el ítem 3 debido a la capacidad restante ($60 > C$).

Por otro lado, la programación dinámica evalúa todas las combinaciones posibles y determina que la mejor solución es seleccionar los ítems 2 y 3, alcanzando un ingreso óptimo de 220 sin exceder la capacidad C .

Pseudopolinomialidad

El algoritmo de programación dinámica tiene complejidad $O(n \cdot C)$. Se considera pseudopolinomial porque depende del valor numérico de la capacidad C y no del número de bits necesarios para representarla. En este proyecto, el valor de C es relativamente pequeño, por lo que el algoritmo resulta eficiente y práctico. Sin embargo, para capacidades extremadamente grandes, el tamaño de la tabla crecería demasiado y el algoritmo dejaría de ser viable.

6. Integración del Pipeline

1. Carga y limpieza del CSV.
2. Generación del grafo (criterio de coincidencia y pesos).
3. Módulo A: ordenar nodos por TotalCharges; responder consultas mediante búsqueda binaria.
4. Módulo B: aplicar Kruskal -> obtener MST; su peso total se usa como capacidad C para la mochila.
5. Módulo C: ejecutar DP con C y con valores = ingresos estimados por solicitud; obtener conjunto óptimo de clientes a servir.
6. Salida final: informe con estadísticas, tablas de tiempos, MST y solución de la mochila.

7. Conclusiones

En este proyecto se pudieron comparar diferentes paradigmas de programación usando el mismo conjunto de datos, lo que ayudó a entender mejor las ventajas y limitaciones de cada uno.

El enfoque de Divide y Vencerás demostró ser eficiente para el ordenamiento de datos, ya que mantiene una complejidad estable y conserva correctamente el orden cuando existen valores repetidos.

El paradigma Codicioso funcionó muy bien para encontrar el árbol de expansión mínima, porque en ese problema tomar la mejor decisión local sí lleva a una solución óptima. Sin embargo, también se comprobó que este enfoque no siempre garantiza la mejor respuesta, como ocurrió en el problema de la mochila 0-1.

Por otro lado, la Programación Dinámica fue el método más costoso computacionalmente, pero permitió encontrar la solución óptima exacta para la asignación de ancho de banda.

Finalmente, trabajar con datos reales hizo que el desarrollo fuera más retador, ya que fue necesario manejar casos como valores nulos, empates y distribuciones irregulares. Esto permitió entender mejor cómo aplicar los algoritmos en situaciones más cercanas a un problema real.

8. Herramientas utilizadas

<https://chatgpt.com/share/69fe702b-40b0-83e9-8318-f41f4e9fc323>

prompt para mejorar nuestro código y performance del módulo C:

revisa mi knapsack.cpp y .hpp y mira si mi código está correctamente implementado o si encuentras errores u oportunidades de mejora y hazmelo saber

2. Corrección Algorítmica (Está muy bien implementado 🍌)

- Programación Dinámica (Mochila 0-1):** Tu función `solveKnapsack` está perfectamente implementada. El uso de tabulación (*bottom-up*) está correcto y el proceso de *backtracking* para recuperar qué ítems se seleccionaron funciona exactamente como debería.
- Manejo de Casos Especiales:** Noté que en `writeKnapsackOutput` manejas adecuadamente el caso donde ninguna solicitud es seleccionada si el peso excede la capacidad. Esto habla muy bien de la solidez del código.

3. Oportunidades de Mejora y Rendimiento (Ya aplicadas 🚀)

El bloque de código que se encarga de buscar un contraejemplo (`findCounterexample`) utiliza un triple ciclo anidado para encontrar todas las combinaciones de 3 elementos y dentro un bucle adicional de capacidades (`cap`). Había un par de detalles en este sector que causarían que la función fuera **extremadamente lenta** al procesar millones de iteraciones:

- Asignación constante de memoria en el ciclo interno:** Dentro de tu iteración más profunda (`for cap = minW ...`) estabas declarando `vector<string> gSel, dpSel;`. Esto obligaba al programa a reservar y liberar memoria dinámica millones de veces (literalmente en cada `cap` de cada combinación), lo que es una penalización de rendimiento enorme. Lo he movido **fuera** de ese bucle interno y he añadido un `.reserve(3)` para reutilizar la misma memoria, limpiando los vectores (`.clear()`) en cada pasada.
- Ordenamiento redundante:** Dentro de tu función `runGreedy` se estaba ordenando el arreglo de 3 elementos usando `std::sort()`. Al llamarse a esta función dentro del bucle de capacidades, el programa estaba ordenando *los mismos 3 elementos una y otra vez* por cada `cap` intentada. He movido el ordenamiento una capa más arriba, en `findCounterexample`, para que los 3 elementos se ordenen **una sola vez por combinación**, pasándolos ya ordenados a `runGreedy`.

Con estos cambios aplicados, la búsqueda de contraejemplos debería ser ahora un orden de magnitud más rápida.

En resumen: tu lógica está muy bien armada y los algoritmos funcionan como se espera. ¡Solo faltaban esos pequeños ajustes de sintaxis y performance! Si tienes alguna duda sobre los cambios que apliqué o quieres que revisemos otra parte de tu proyecto, no dudes en decirme.

explicame por qué hiciste esos cambios del performance más a fondo para comprender tus decisiones

Thought for 5s

¡Claro que sí! Es una excelente pregunta. Cuando programamos algoritmos que analizan combinaciones (fuerza bruta o búsquedas exhaustivas), entender qué sucede "detrás de escena" en la computadora hace toda la diferencia entre un programa que tarda segundos y uno que tarda horas.

Para comprender el impacto, primero veamos la escala del problema analizando tus ciclos `for`:

- Tienes un ciclo triple `(i, j, k)` para elegir 3 elementos de un total de 50. Esto significa que hay exactamente **19,600 combinaciones posibles** de 3 solicitudes $\frac{50 \times 49 \times 48}{3 \times 2 \times 1}$.
- Por cada una de esas 19,600 combinaciones, haces un cuarto ciclo `for (int cap = minW; cap < totalW; cap++)`. Los pesos (`weight`) en tu código provienen de `totalCharges`, los cuales pueden ser números de miles de unidades. Esto significa que para **cada combinación**, este ciclo interno podría ejecutarse fácilmente cientos o miles de veces.

Si multiplicamos, el código que está dentro del ciclo `cap` podría llegar a ejecutarse **decenas de millones de veces**. Cuando un bloque de código se ejecuta tantas veces, cualquier operación pequeña (incluso una que normalmente tardaría milisegundos) se convierte en un cuello de botella enorme.

Aquí te explico detalladamente las dos cosas que extrajimos de ese bloque y por qué:

- El problema de la "Asignación Dinámica de Memoria" (Los vectores)**
Cómo estaba antes: Dentro del ciclo `for(int cap...)` tenías esta línea: `vector<string> gSel, dpSel;`. En C++, cuando creas un vector, este pide memoria al Sistema Operativo (memoria *Heap* o *montículo*). Pedirle memoria al Sistema Operativo es una de las operaciones más lentas que puede hacer un programa porque requiere salir de tu código, ir al kernel del SO, buscar un bloque libre, registrarlo y devolvértelo. Al terminar la iteración del bucle, el vector se destruye y devuelve esa memoria.
 Al estar dentro del ciclo, tu programa estaba **pidiendo y devolviendo memoria al Sistema Operativo decenas de millones de veces**.
La solución: Lo que hicimos fue declarar los vectores **antes** de empezar el ciclo `cap` y usar `.reserve(3)` (ya que sabemos que máximo guardarán 3 ítems).

9. Referencias

Kaggle, Telco Customer Churn dataset –

<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. Introduction to Algorithms, 3ª ed., MIT Press, 2009.

Material de la asignatura de Análisis y diseño de algoritmos (diapositivas).

10. Roles del Equipo

integrante	Módulo	Contribución principal
Valentina zapata	A	Implementación de MergeSort y búsqueda binaria; generación de tablas de tiempos y consultas.
Sofia Gallo	B	Desarrollo de Kruskal con Union-Find; cálculo del MST y documentación del Lema del ciclo.
Mariana Carrasquilla	C	Formulación de la recurrencia de la mochila 0-1; codificación DP y backtracking; análisis del contraejemplo greedy.